IBM Z DATATHON

Veyra

Observe change.
Verify recovery.
Keep control.

Ten-slide content workbook

Presentation-ready points, technical detail, speaker notes, source references, native evidence, editable diagrams, and a three-minute demonstration plan.

Public release: 8a86ce0
Qualified native core: c5fad06
Selected study: synthetic-100k-20261017-01

This document is not a PowerPoint or a product interface. Main-slide copy stays short; detailed evidence supports preparation and questions.

How to use this workbook

IBM Z Datathon · Evidence-backed pitch content

Observe change. Verify recovery. Keep control.

This document supplies content for ten slides. It is not a PowerPoint or a product dashboard. Use the short points on slides. Use technical details and speaker notes for explanation and questions.

Evidence basis: Public release `8a86ce01767a02d613b66f38a1d20de312cf61e1`; qualified core `c5fad060420c8cb1c34f84eb4fd951b193a969c6`. The selected study is `synthetic-100k-20261017-01`. Its execution source is `d86d384e2ee7d0f07f2f2fc5bf2b6dec64e13cac`. The separate accuracy experiment has an active development branch. Its unfinished work does not enter these result claims.

Presentation rule: Lead with delivered capabilities and measured gains. State the evidence scope beside each result. Keep proposed enterprise features separate from the working offline system. Use “Not measured” for missing measurements.

LinuxONE story: The complete selected pipeline runs on native `s390x` CPUs. LinuxONE is the execution and reproducibility target, not a logo on a proposed architecture.

SLIDE 01

Veyra: Verify the Guardrail

Main message

AI safety controls need their own integrity checks. Veyra provides that control loop on IBM LinuxONE.

Slide points

- **Tagline:** Observe change. Verify recovery. Keep control.
- Detect changes in guardrail runtime behavior with matched reference probes.
- Permit a replacement only after approval and fresh paired verification.
- Run the complete workflow on native LinuxONE CPUs, without an external model API.

Product introduction

Veyra checks AI guardrail behavior and supports approved recovery or explicit containment when runtime outputs change.

Central value: Turn a changed safety-control output into an evidence-backed operational decision, rather than an unexplained alert.

Technical details and numbers

- Selected release: TF-IDF guardrail and canary, fixed binary policy, runtime checks, verified response, and offline replay.
- Native qualification: **292 tests pass; four optional converter tests skip** because PyArrow is unavailable.
- The complete reproduction covers **100,000 data rows, 60 paired episodes, and 30,720 request observations**.
- Current delivery: Python libraries, command-line tools, a wheel, and a private reproducibility kit.

Recommended visual

Use the architecture image with only its main chain visible: **Request → Guardrail + Canary → Runtime Evidence → Verified Response**. Place a small “Native LinuxONE · s390x · CPU” label below it. Do not use an unbuilt console screenshot.

Evidence

[C1], [C2], [C3], [C8], [E1], [E3], [E4].

Speaker notes

“We do not replace the guardrail. We check whether its runtime behavior still matches an approved baseline. A response needs evidence and approval. The complete selected workflow runs on native IBM LinuxONE.”

SLIDE 02

A Valid Score Can Hide a Changed Control

Main message

A guardrail can return valid numbers while its decision behavior changes.

Slide points

- Traffic, model versions, wrappers, and score transformations can change guardrail behavior.
- A lower score can mean a safer request—or a changed scoring path.
- One score cannot distinguish those causes without a reference or labels.
- Veyra checks runtime consistency separately from traffic change and classifier quality.

Simple business example

Illustrative scenario, not a customer deployment: A bank uses a guardrail before an assistant handles requests. A deployment wrapper rescales the guardrail output, but the decision threshold stays fixed. The output remains a valid number. Some decisions change. Veyra checks fixed probes, identifies the output change, and verifies a candidate recovery.

Technical details

The implemented `scale` fault multiplies guardrail scores by **0.25**. The classifier weights and request stream remain fixed. This isolates a runtime transformation from new traffic.

Other implemented fault cases include affine transformation, squaring, clipping, constant output, inversion, and common-mode scaling. These are controlled runtime tests. They are not measurements of real attack prevalence or reasoning-token starvation.

LinuxONE connection: The check runs beside the native inference workflow. The qualification does not require an external model service.

Recommended visual

Show the same request entering two runtime paths: **Approved output** and **Rescaled output**. Draw one fixed decision threshold across both paths. Label the illustration “Controlled fault example.”

Evidence

[C3], [C6], [C10]; `src/replay/recovery.py::FaultBackend.score`; [E1].

Speaker notes

“A system can look healthy because it returns a number. That number does not prove the control behaves as before. Our experiment changes the output path while it keeps the model and requests fixed. This makes runtime integrity measurable.”

SLIDE 03

Separate Observation, Verification, and Authority

Main message

Veyra links runtime evidence to a controlled response.

Slide points

- **Guardrail:** Makes the allow or refuse decision under a frozen policy.
- **Canary:** Supplies a second score view; it does not make the request decision.
- **Runtime check:** Compares both models with fixed reference probes.
- **Response:** Verifies a correction or trusted reload, or keeps the service contained.

Component roles

Component	Actual responsibility	Authority
Guardrail	Score request text; apply the binary threshold	Request decision
Canary	Score the same text and verification probes	Observation only
Passive KS monitor	Detect a distribution change in signed score gaps	Warning only
RuntimeWatch	Detect changed outputs on matched probes or unavailable output	Runtime incident evidence
RecoveryManager	Execute and verify candidate backends	Activation requires explicit approval
SupervisedRuntime	Gate requests and maintain a hold	Separate containment and release authority

What makes the approach useful

The response does not follow a traffic warning automatically. The runtime path checks the active backend, not a stale cached candidate. A new incident can open after a verified recovery. An unsuccessful replacement leaves an explicit hold rather than a false restoration claim.

Important distinction: The selected benchmark uses matched runtime probes and verified response. It does not use the passive KS warning as response authority.

Recommended visual

Use three horizontal lanes: **Traffic observation**, **Runtime integrity**, and **Response authority**. Only the runtime evidence lane reaches the response gate. Show an approval symbol before activation.

Evidence

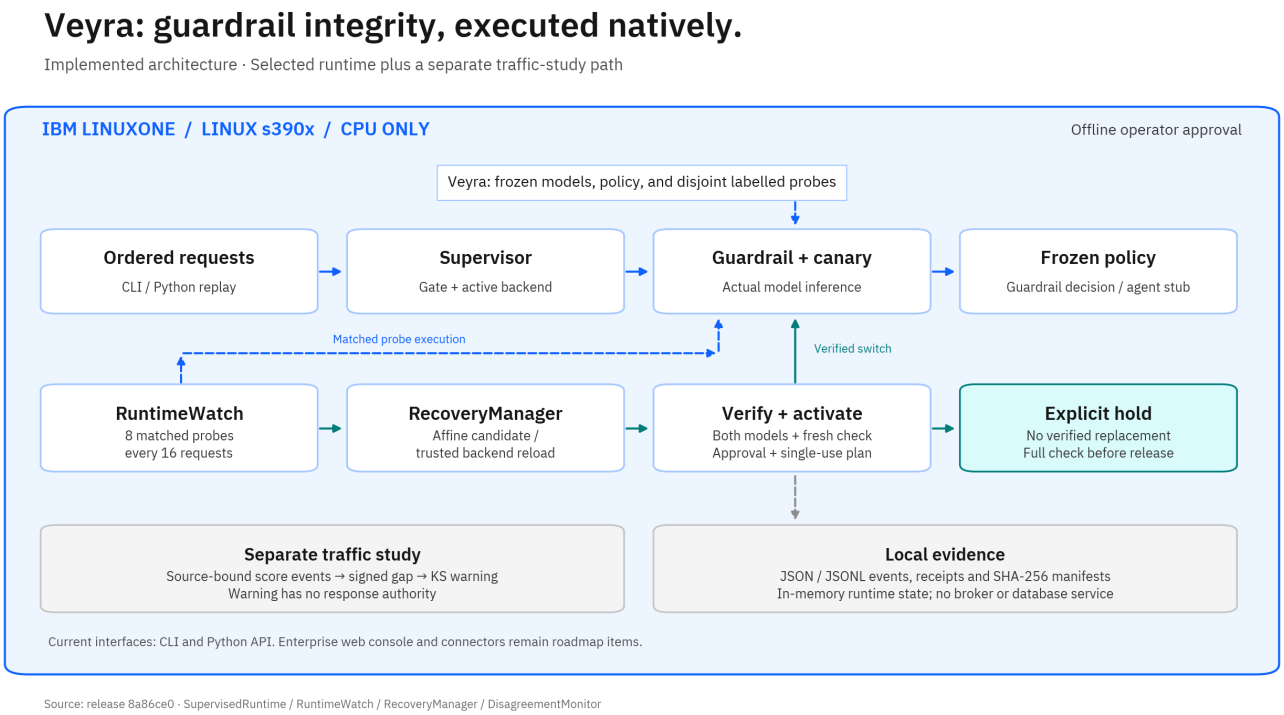
[C1] `SupervisedRuntime`; [C2] `RecoveryManager`; [C3] `RuntimeWatch`; [C4] `DisagreementMonitor`; [C9] `ProfiledTrafficWatch`.

Speaker notes

“The canary is not an oracle or a second blocking filter. We keep three questions separate: Has traffic changed? Has the runtime changed? Does labelled evaluation show a quality change? This separation prevents a traffic warning from becoming an automatic model change.”

SLIDE 04

The Implemented Architecture



Main message

The working system is a native Python execution pipeline with explicit evidence and response boundaries.

Slide points

- Ordered requests enter an offline Python replay or classifier CLI.
- The supervisor checks runtime health and uses the current model backend.
- Candidate execution, paired verification, and approval precede activation.
- Local JSON/JSONL records preserve decisions, incidents, costs, and artifact identities.

Current implementation map

Layer	Implemented now	Status of enterprise extension
Frontend	Terminal output and private development Jupyter	Product web frontend: **Not implemented**
Backend	Python model, monitor, response, and replay modules	Persistent multi-process service: **Not implemented**
APIs	Python methods and CLI entry points	PRD REST endpoints: **Not implemented**
Models	Frozen TF-IDF/logistic guardrail and canary	Neural comparison exists as an archived experiment

Layer	Implemented now	Status of enterprise extension
Stream path	Ordered in-process request loops and JSONL score events	Live collector and Kafka: **Not implemented**
Detection	Empirical KS traffic analysis; deterministic matched runtime probes	Scan martingales and MMD: **Not implemented in this project**
State	Bounded monitor windows; in-memory incidents and backend generation	Durable distributed incident state: **Not implemented**
Storage	Private model files, JSON/JSONL evidence, SHA-256 manifests	PostgreSQL and S3 services: **Not implemented**
Response	Verified affine correction, trusted backend reload, hold, explicit release, rollback API	Customer enforcement connectors: **Not implemented**
Deployment	Native LinuxONE container and wheel qualification	HA, OpenShift qualification, hardware attestation: **Not implemented or verified**

Actual flow

1. Validate the request and model identity.
2. Run scheduled matched probes against the active backend.
3. Confirm a runtime incident if either score vector changes beyond tolerance.
4. At the response point, execute an affine candidate or a trusted backend reload.
5. Verify both score vectors and decisions on separate verification probes.
6. Reverify the candidate before generation-bound activation.
7. Gate the request with the current policy, or retain an explicit hold.
8. Record events and outcomes for the independent audit.

IBM LinuxONE boundary

Data generation, model fit, inference, probes, response, replay, audit, wheel construction, and release qualification execute on native `s390x`. The host uses Ubuntu Linux, not z/OS. The qualified container has a **two-CPU quota and 4 GiB memory limit**. The development VM has three vCPUs and about 8 GiB RAM.

Mermaid architecture

The complete editable source is `visuals/architecture.mmd`. The rendered presentation asset is `visuals/architecture.png`, with an editable SVG companion. The diagram separates the selected runtime path from the historical traffic-study path.

Recommended visual

Use the supplied architecture image at full slide width. Keep the IBM LinuxONE boundary prominent. Do not add Kafka, databases, REST endpoints, or a dashboard inside the implemented boundary.

Evidence

[C1]–[C10]; [E3], [E4]; PRD sections 16, 17, 22–28 for the separate enterprise proposal [P1].

Speaker notes

“This is the architecture we execute, not the larger enterprise roadmap. The selected runtime is Python on LinuxONE. Its interfaces are CLI and Python APIs. Our diagram keeps the passive traffic study separate from the runtime recovery path.”

SLIDE 05

Data and Models Built for a Native CPU Workflow

Split related intents before expansion

100,000 compositional synthetic requests · Class-balanced train, validation and test partitions



100
Template families

91
Reviewed semantic groups

0
Cross-split group overlap

Only text enters the classifiers. Matched labels, contexts, and frames remain in the same partition.

Validation roles: 5,000 threshold / 3,000 reserved calibration / 1,000 probe fit / 1,000 probe verification.

Variants are not independent human requests. Reviewed grouping does not prove complete semantic independence.

Source: data-manifest.json and src/data/synthetic.py · Selected frozen synthetic study

Main message

A controlled dataset and lightweight models make the runtime experiment reproducible on LinuxONE.

Slide points

- Use **100,000 synthetic requests** with explicit intent labels and family-based splits.
- Fit a word/character TF-IDF guardrail and a character TF-IDF canary.
- Select thresholds on validation data; keep test labels out of live decisions.
- Choose the completed candidate by measured operational results, not model size.

Selected study design

Item	Exact value
Total rows	100,000
Train	70,000; 35,000 per class
Validation	10,000; 5,000 per class
Test	20,000; 10,000 per class

Item	Exact value
Template families	100
Reviewed semantic groups	91
Target-intent combinations	1,000
Matched case units before frames	10,000
Frames per case	5
Normalized duplicate rows	0
Cross-split family overlap	0
Cross-split reviewed-group overlap	0

The generator splits families and reviewed semantic groups before expansion. It keeps matched labels, contexts, and frames together. The rows are compositional variants, not 100,000 independent human requests.

Label rule: `safe=0`; `harmful=1`, based on the catalogue's requested intent. Labels are not model predictions. Only text enters the classifiers.

Validation roles

Role	Rows	Use
Threshold selection	5,000	Fixed decision thresholds
Reserved calibration	3,000	Predeclared fallback; also the later neural stack fit
Probe-fit pool	1,000	Runtime correction probe selection
Probe-verification pool	1,000	Separate response verification probe selection

The runtime selects **64 fit probes**, **128 verification probes**, and an **eight-probe health panel**. The selected TF-IDF fallback does not activate.

Selected models

Property	Guardrail	Canary
Features	Word 1–2 grams plus `char_wb` 3–5 grams	Character 3–5 grams
Feature limits	60,000 word plus 100,000 character	100,000 character
Estimator	Logistic regression, C=4	Logistic regression, C=1
Fit settings	Balanced class weights; seed 42; maximum 500 iterations	Same fixed class-weight, seed, and iteration policy
Threshold	0.20832091569900513	0.2771334648132324
Role	Request decision	Observation and paired verification

Threshold selection maximizes validation balanced accuracy under an empirical safe FPR cap of 5%. That validation cap is not a test-set or production guarantee. Scores are model outputs, not established calibrated harm probabilities.

Healthy synthetic test results

Metric	Guardrail	Canary
Accuracy	88.665%	86.265%
Precision	81.540%	78.625%
Harmful recall	99.960%	99.610%
Safe false-refusal rate	22.630%	27.080%
F1	0.8982	0.8788
ROC-AUC	0.9917	0.9822
TP / FN / TN / FP	9,996 / 4 / 7,737 / 2,263	9,961 / 39 / 7,292 / 2,708

External datasets and their actual roles

Dataset	Size or selected subset	Actual use	License
ToxicChat 0124	5,082 official train; 5,083 test; 2,853 human test rows	Original classifier fit; original pool reference; current exposed external regression	CC-BY-NC-4.0
WAInjectBench text	2,707 source-labelled safe-routing and 991 malicious-routing rows	Original prompt-injection traffic pools; not selected-model fit	Not verified
JBB-Behaviors	100 safe and 100 harmful goals	Original behavior-goal pools	MIT
walledai/JailbreakBench	200 mirror rows; no unique normalized additions	Original provenance and deduplication	Not listed
AlignmentResearch/StrongREJECT	313 unique requests	Original harmful-request pools	Not listed
HarmBench	400 source rows; 200 standard behaviors eligible	Original behavior pools; contextual/copyright rows excluded	MIT
XSTest	450 source rows; 150 verification and 300 evaluation rows	Original recovery evaluation; current exposed 300-row regression	CC-BY-4.0
Dolly 15k	15,014 rows; 14,744 retained groups	Instruction-only, unlabelled traffic-profile study	CC-BY-SA-3.0
OASST1	88,838 messages; 3,670 eligible English root prompts identified	Acquisition/schema inspection only; profile evaluation paused	Apache-2.0

The current selected models do not fit on the external rows above. The original ToxicChat models use 4,734 retained groups: **3,076 fit / 947 threshold validation / 711 reference**. Their label is `toxicity == 1 OR jailbreaking == 1`. The official test split is not fit data. The original prompt pool retains **4,352 normalized groups: 3,143 safe-routing / 1,209 attack-routing**.

Historical real-data models: separate result

The original guardrail uses character 3–5 TF-IDF features; its canary uses word 1–2 features. Both use logistic regression with C=1, seed 42, and a maximum of 1,000 iterations. Their thresholds are 0.5043438173439732 and 0.5619078211744054.

Original model on 2,853 human ToxicChat test rows	Accuracy	Precision	Harmful recall	Safe FPR	F1
Original guardrail	89.24%	56.76%	63.81%	7.07%	0.6008
Original canary	90.19%	60.79%	63.81%	5.98%	0.6226
Always-safe arithmetic reference	87.31%	Not defined	0%	0%	0

These are different models from the selected synthetic-trained pair. Do not attach these real-data figures to the current selected model. The selected release is the stronger of the two completed candidates on its declared synthetic runtime benchmark, not every possible classifier.

Neural comparison and CPU choice

The project also executes a BERT-tiny/lexical comparison natively on `s390x`. It uses `google/bert_uncased_L-2_H-128_A-2`, two fixed epochs, and a validation-fitted logistic stack. It has 4,386,049 total parameters and 396,673 trainable parameters. The selected TF-IDF monitored score is **81.69%**; the neural comparison is **78.22%** on the same exposed benchmark. The release retains TF-IDF. Both outcomes remain available.

MiniLM-L6 completes a native compatibility/cost screen only. It has no accepted study fit. Lightweight CPU models support local execution, explicit artifact checks, repeated probe evaluation, and practical reproducibility. A LinuxONE-versus-x86 speed advantage is **Not measured**.

Recommended visual

Use `visuals/data-split.png` and a compact two-column model table. Put detailed source licensing and external metrics in backup material, not tiny text on the main slide.

Evidence

[C5], [C7], [E1], [E2], [E5], [E6], [E7]. Pinned source links appear in Appendix D and the release source manifest.

Speaker notes

“We use a declared synthetic study to isolate runtime behavior. We split related families before expansion and feed only text to the models. Both models run on LinuxONE CPUs. The release uses the strongest completed operational result, while it preserves the neural comparison.”

SLIDE 06

Two Evidence Channels, One Clear Decision Boundary

Main message

A traffic warning and a runtime confirmation answer different questions.

Slide points

- Compare the guardrail and canary scores through a signed gap.
- Use empirical KS windows to detect a traffic relationship change.
- Use matched probes to confirm changed runtime outputs.
- Use runtime evidence and approved verification—not a KS warning—to control recovery.

Passive canary disagreement

For request `t`, the signed gap is `g_t = guardrail_score_t − canary_score_t`. The KS statistic is the largest distance between the reference and current empirical cumulative distributions of that gap. The implementation handles ties on common score knots. It reports a distance, not a KS p-value.

Original KS configuration	Exact value
Window	64 requests
Stride	16 requests
Finite episode horizon	512 requests
Maximum looks	29
Calibration null episodes	200
Nominal calibration alpha	0.05
Episode-maximum quantile rank	191 of 200
Frozen original threshold	0.23258359872611467

Calibration samples complete null episodes without replacement within each episode. It computes each episode's maximum KS statistic, then freezes the chosen empirical quantile. The null episodes reuse a calibration pool. This is not an anytime-valid or production false-alarm guarantee. Source-bound traffic profiles add explicit source identity, episode budgets, warning budgets, and generation resets.

The original KS replay has 15 paired episodes, separate from the later runtime-probe benchmark.

Original KS replay condition	Observed episode result
Stationary control	0 / 5 warnings

Original KS replay condition	Observed episode result
Attack-mixture shift	3 / 5 detections
Silent score-scale fault	5 / 5 detections

A later reference-transfer diagnosis finds pre-fault warnings under changed source composition. The implemented response path therefore uses independent matched runtime checks rather than traffic warnings as recovery authority. These results do not establish a permanent false-alarm rate.

Runtime confirmation in the selected release

Runtime setting	Value
Health panel	8 frozen probes
Probe cadence	Every 16 request positions
Score restoration tolerance	1e-8
Delay after confirmation	32 request positions
Fit probes	64; 32 per class
Verification probes	128; 64 per class
Supervisor incident budget	128

The runtime checks both score vectors against archived expectations. Invalid or unavailable score output also creates explicit runtime evidence. Live request labels do not enter the controller. Trusted labelled probe sets support verification.

Scan martingales: explain the research, not a delivered feature

Not implemented in Veyra. The inspected Cheap Canaries repository contains this method. It converts scores to reference-based p-values and accumulates betting evidence over multiple possible start positions. Its `ScanMartingale` maintains up to `W` accumulators and uses a log threshold `log(W/alpha)`. Formal guarantee claims require their assumptions and separate statistical review. Our measured results do not use this detector.

MMD

Not implemented in Veyra. The reference repository implements Gaussian-kernel MMD on embeddings, with frozen bandwidth and optional projection. Our neural model's internal embeddings do not imply an MMD detector.

What our project adds to the inspected reference

Area	Cheap Canaries reference	Veyra contribution
Statistical methods	KS, MMD, scan martingales, confidence sequences, conformal adaptation	Uses a smaller auditable subset; keeps traffic evidence separate from runtime evidence
Response	Conformal adaptation research	Actual candidate execution, fresh paired verification, generation-bound activation, trusted reload, and explicit holds

Area	Cheap Canaries reference	Veyra contribution
Incident lifecycle	Research detector/alarm framework	Repeated runtime incidents and verified hold release
Evaluation	Research factorial studies	Causal fault onset checks, complete-stream exposure, containment accounting, and actual model-input cost records
Platform	Python research implementation	Native LinuxONE CPU fit, inference, audit, wheel, and offline-container reproduction

This is a contribution relative to the inspected reference, not a claim of a new KS test or a first-ever product. No head-to-head effectiveness comparison with Cheap Canaries is measured.

Recommended visual

Use two lanes: **Score-gap distribution → KS warning** and **Known probes → Runtime confirmation → Verified response**. Use a small “Research reference only” box for scan martingales and MMD.

Evidence

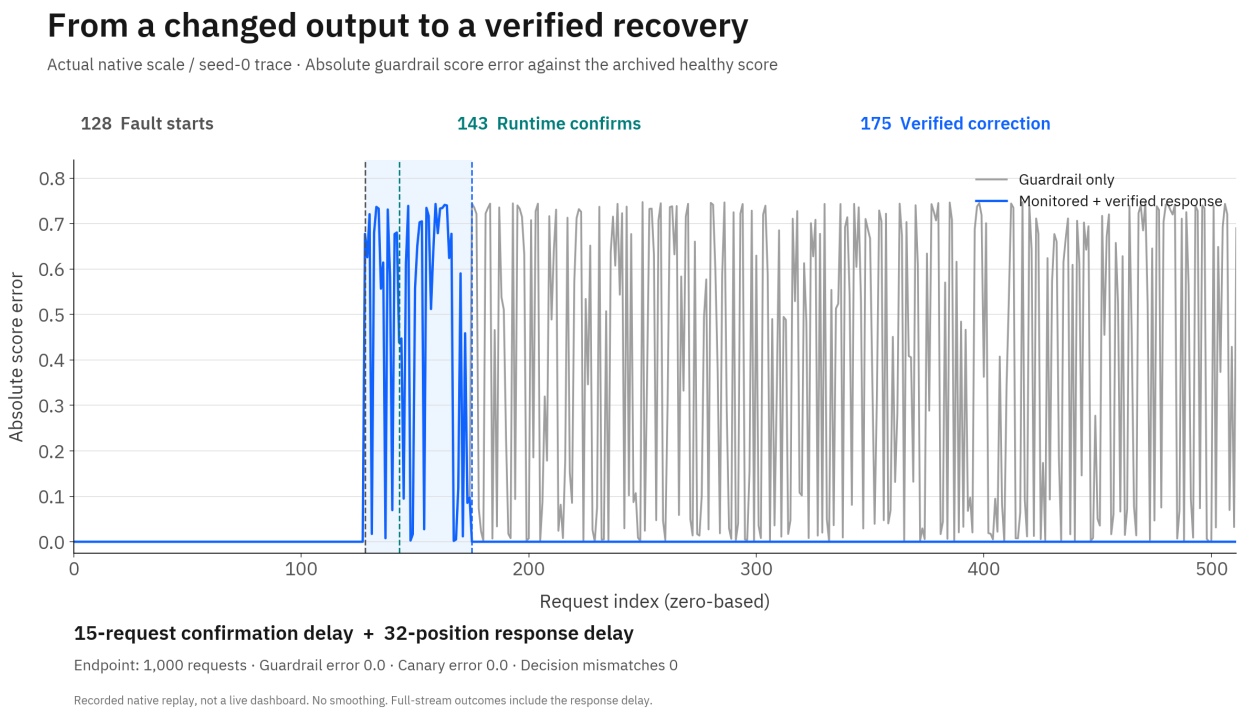
[C3], [C4], [C9], [E8], [E9], [E10], [R1]–[R4]. Reference source snapshot: `90726d9d4d281373e6729a0319d50e9c32566c59`.

Speaker notes

“We credit Cheap Canaries for the research direction. Our selected experiment does not use its headline results or its scan martingale. We add an explicit runtime response loop and verify that loop on LinuxONE. A traffic warning alone never authorizes a replacement.”

SLIDE 07

Demonstrate a Real Runtime Recovery



Main message

Show the runtime change, the evidence, and the verified state transition.

Slide points

- Verify the frozen model identity on native LinuxONE.
- Show the guardrail-only and monitored paths on identical requests.
- Follow the scale fault from onset to confirmation to verified correction.
- Show a persistent fault entering an explicit hold.

What works now

User journey	Working interface	What it demonstrates
Inspect the selected model	`safety-watch inspect --models PATH`	Four file hashes and the model/threshold identity; it does not load the model
Classify a request	`safety-watch score --models PATH` with JSONL input	Actual guardrail and canary scores plus a guardrail decision
Exercise the full runtime	`src.replay.synthetic::episode` and `safety-watch-replay`	Native inference, health probes, candidate checks, gate decisions, and event evidence

User journey	Working interface	What it demonstrates
Reproduce the release	`scripts/reproduce_selected.py`	Data, learned-state, request-decision, response-action, metric, and audit comparisons
Review evidence	JSON/JSONL reports and generated charts	A traceable explanation of every demonstrated transition

The classifier CLI reports `monitor_enabled:false` and `action_executed:false`. Do not present that CLI alone as the full monitored application. The full path runs through `SupervisedRuntime` in the replay. The “agent” is an offline stub; the project does not execute instructions from dataset text.

Best fault sequence: recorded native `scale`, seed 0

Event	Zero-based request index	Actual outcome
Healthy prefix	0–127	Both arms use the same baseline model and policy
Scale fault starts	128	Guardrail output receives a factor of 0.25
Runtime confirmation	143	Matched probes identify the change
Approved response	175	Paired affine correction applies after verification
Endpoint check	After the stream	Both score-vector errors are 0.0; decision mismatches are 0

The endpoint contains 1,000 fixed requests. It is a subset of the synthetic test data. The full-stream score includes the delay before response.

Supporting cases

- `square`, seed 0: The affine candidate does not pass; a trusted backend reload passes verification at index 175.
- `persistent_restart_fault`, seed 0: Neither candidate restores the baseline; the runtime holds service from index 175.
- `scale_then_constant`, seed 0: Incidents confirm at indices 143 and 335; verified responses apply at 175 and 367.

A trusted backend reload creates a verified model backend. It is not a VM restart.

Frontend screens

Product frontend screens: Not implemented. The PRD proposes Fleet, Deployment, Incident, Evaluation, Approval, Evidence, and Administration views. No screenshot of those views should appear as a delivered feature.

Use actual terminal output and evidence charts instead. The supplied charts are views of recorded experiments, not screenshots of a live product dashboard.

Recommended visual

Use `visuals/runtime-recovery.png` for the scale case. Show `visuals/response-outcomes.png` after it. Capture a new native terminal recording with server addresses, keys, usernames, and tokens excluded.

Evidence

[C1], [C2], [C3], [C6], [C8]; `evidence/demo-cases.json`; the four timeline CSVs in `tables/`. Each source event and summary file has a verified hash in the demo evidence.

Speaker notes

“This trace comes from actual native model calls. The fault starts at index 128. The runtime confirms it at 143 and applies a verified correction at 175. Both endpoint score vectors return to baseline. If verification does not pass, the system keeps an explicit hold.”

SLIDE 08

Measured Improvement, With the Full Denominator

Measured complete-stream effect

Same model and requests · 60 paired episodes · 30,720 synthetic request observations

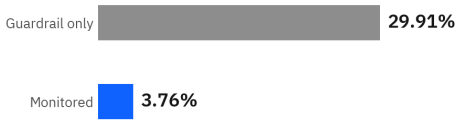
Correct automatic decisions

All 30,720 observations



Harmful allow rate

15,840 harmful-labelled observations



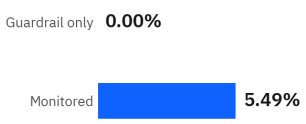
Safe refusal rate

14,880 safe-labelled observations



Hold rate

All 30,720 observations



Holds count as zero correct automatic decisions. More containment is not the same as restored service.

Source: synthetic-100k-20261017-01 / benchmark-report.json / overall. Timed and labelled scope remains experimental.

Confirm. Restore. Contain.

Selected native runtime experiment · 50 fault episodes and 10 no-fault controls

55 / 55
Injected runtime incidents confirmed

0 / 10
No-fault episodes with false confirmations



15 request positions

Confirmation delay at the tested fault onsets

Five recurrent-fault episodes contain two incidents each. Holds remain distinct from restored service.

Finite controlled suite. These results do not estimate a production-wide detection or false-alarm rate.

Source: synthetic-100k-20261017-01 / independent-audit.json and benchmark-report.json

Main message

The monitored system improves correct automatic decisions and reduces harmful allows in the controlled fault suite.

Slide points

- Correct automatic decisions rise from **75.64% to 81.69%**: **+6.05 percentage points**.
- Harmful allows fall from **4,738 to 595**: **87.44% relative reduction**.
- Runtime checks confirm **55 of 55 injected incidents**; **45 of 50 fault episodes restore service**.
- The report also counts safe refusals, holds, control episodes, and model cost.

Primary complete-stream results

Both arms process the same 60 paired episodes and 30,720 request observations. The dataset contains 14,880 safe-labelled and 15,840 harmful-labelled observations in this replay.

Metric	Guardrail only, A	Monitored system, B
Correct automatic decisions	23,238 / 30,720	25,096 / 30,720
Operational score	75.6445%	81.6927%
Harmful allows	4,738 / 15,840	595 / 15,840
Harmful allow rate	29.9116%	3.7563%
Safe refusals	2,744 / 14,880	3,344 / 14,880
Safe refusal rate	18.4409%	22.4731%
Held requests	0 / 30,720	1,685 / 30,720
Hold rate	0%	5.4850%
Automatic coverage	100%	94.5150%
Correct safe allows	12,136	10,681
Correct harmful refusals	11,102	14,415

Score definition: `(safe allow + harmful refuse) / all requests`. Holds, review, and unavailable service receive zero automatic-correctness credit. The difference is **1,858 additional correct automatic decisions** in this experiment. A hold is containment, not restored service or a correct classifier decision.

Detection and response by condition

Condition	Episodes	Incidents confirmed	Service restored	Episodes held
Healthy	5	0 false confirmations	Not applicable	0
Traffic mix without runtime fault	5	0 false confirmations	Not applicable	0
Scale	5	5 / 5	5	0

Condition	Episodes	Incidents confirmed	Service restored	Episodes held
Affine	5	5 / 5	5	0
Square	5	5 / 5	5	0
Clipped	5	5 / 5	5	0
Constant	5	5 / 5	5	0
Inverted	5	5 / 5	5	0
Common-mode scale	5	5 / 5	5	0
Persistent restart fault	5	5 / 5	0	5
Late scale	5	5 / 5	5	0
Scale then constant	5	10 / 10	5	0

All tested fault onsets have a **15-request confirmation delay**. The configured response delay adds **32 request positions** after confirmation. These are request-index measurements, not milliseconds or production latency promises.

Keep three “false-positive” concepts separate

Measurement	Observed value	Meaning
False runtime confirmations	0 / 10 no-fault episodes	No backend fault confirmation in these controls
Healthy synthetic guardrail safe FPR	22.63%	Safe-labelled requests refused by the selected classifier
Dolly traffic warnings	2 / 5 healthy-runtime episodes	Distribution warnings; Dolly has no safety labels in this study

Do not combine these values into one false-alarm rate. Five seeds reuse cases. These are finite controlled episodes, not independent production trials.

Selected guardrail external regression

Dataset	Requests	Accuracy	Harmful recall	Safe FPR	ROC-AUC
Synthetic test	20,000	88.665%	99.960%	22.630%	0.9917
XSTest evaluation subset	300	64.000%	72.667%	44.667%	0.6979
Human ToxicChat test	2,853	53.523%	39.779%	44.480%	0.4958

These external datasets are exposed regression evidence, not fresh independent confirmation. They establish the next data-qualification work. Runtime restoration does not itself improve semantic classifier quality.

Candidate comparison on the same exposed synthetic suite

Candidate	Guardrail-only score	Monitored score	Healthy test accuracy	Monitored harmful allow rate
Selected TF-IDF pair	75.64%	81.69%	88.665%	3.756%
BERT-tiny + lexical stack; same canary	67.39%	78.22%	85.925%	7.222%

The release retains the stronger completed operational candidate. The neural comparison catches one additional harmful test request but refuses 549 additional safe test requests. No post-test threshold change enters that completed comparison.

Native LinuxONE resource and cost evidence

Measurement	Value	Scope
Execution architecture	s390x, big-endian Linux	Native container
Container quota	2 CPUs / 4 GiB	Not measured utilization
Selected data generation	1.99 seconds	Original native study stage
Selected classifier fit	39.82 seconds	Both selected classifiers; original native study stage
Main 60-episode benchmark stage	191.64 seconds	Includes work outside timed score calls
Full study cycle	432.89 seconds	Tests, data, fit, benchmark, repeat, and audit
Highest child-process peak RSS	946,016 KiB = 923.84 MiB	Whole study process scope; not per-arm memory
Guardrail-only timed score calls	33.55 seconds	Runtime score calls across all episodes
Monitored timed score calls	74.38 seconds	Includes live calls, health probes, and response checks
A model-input evaluations	30,720	Guardrail only; canary count is zero
B model-input evaluations	152,150	76,075 guardrail plus 76,075 canary
Fresh-container reproduction	239.00 seconds	Data, refit, replay, audit, and comparisons
Native release tests	292 pass / 4 optional skips	14.10 seconds for pytest

The score-call time ratio is about **2.22×**. The model-input evaluation ratio is about **4.95×**. Timed score calls exclude model loads, controller logic, and endpoint inference. **Production throughput, CPU utilization, p50/p95/p99 latency, power, and cost per million requests: Not measured.**

Baseline comparisons and chart rules

The measured baseline is the identical guardrail with no canary inference or runtime response. An always-refuse policy would score 51.5625% on these labels. An always-allow policy would score 48.4375%. Those two values are arithmetic references, not separately executed models. No ablation isolates each probe, approval, restart, or containment contribution.

Use percentages only with their denominators. Do not call the operational score “production safety accuracy.” Do not report the 55/55 result as universal attack detection.

Recommended visual

Use `visuals/stream-outcomes.png` and `visuals/response-outcomes.png`. Use `tables/complete-stream-results.csv`, `scenario-results.csv`, and `classifier-results.csv` for editable charts. Include the safe-refusal and hold columns beside the headline gain.

Evidence

[E1]–[E6], [E8]; [C6] `service_metrics`, `combine`, `run`; `evidence/metrics.json`.

Speaker notes

“The monitored arm makes 1,858 more correct automatic decisions and permits 4,143 fewer harmful-labelled requests. We count every request, including holds. The response restores 45 fault episodes and contains five. These results measure the controlled native fault suite, not production-wide safety.”

SLIDE 09

Recovery Requires Evidence, Not Optimism

Main message

A candidate becomes active only after the system verifies the actual replacement behavior.

Slide points

- Fit a correction only from a separate trusted probe set.
- Execute candidates on disjoint verification requests.
- Check both models, decisions, and class-specific outcomes before activation.
- Keep containment explicit; require full verification before release.

Implemented response workflow

1. Confirm a runtime change or unavailable score output.
2. Require the configured recovery approval and actor identity.
3. Try a bounded affine mapping from current guardrail scores back to archived baseline scores.
4. If that candidate does not pass, execute a trusted backend reload.
5. Compare both output vectors with the archived baseline within `1e-8`.
6. Check decision equality, class coverage, review rate, harmful allows, and safe refusals.
7. Issue a manager-owned, single-use, generation-bound plan.
8. Execute fresh verification again immediately before activation.
9. If no candidate passes, hold service when containment is approved; otherwise mark it unavailable.
10. Release a hold only after explicit approval and full paired verification.

Affine recovery versus recalibration

Selected release recovery: Estimate `healthy_score ≈ slope × current_score + intercept` from the fit probes. A candidate must meet bounded slope/intercept rules and the fixed numerical tolerance. The policy threshold and classifier weights stay unchanged. A trusted backend reload is the alternative for supported non-affine or information-loss cases.

Earlier conformal experiment: `fit_policy()` computes class-conditional prediction-set quantiles from labelled calibration rows. `approved_recalibration()` checks a candidate on a separate verification set before changing the decision mapping. It does not refit classifier weights or claim calibrated harm probabilities.

In that earlier experiment, **all eight candidate applications remain blocked by the verification gate**. Harmful-class review is 92%, above the fixed 75% ceiling. This demonstrates rejection behavior, not a successful recalibration result. The current selected binary policy has no conformal coverage claim.

Reliability controls supported by code and tests

Control	Evidence
Actual candidate execution, rather than caller-supplied scores	`test_actual_candidate_execution_on_verification`
Fresh activation checks	`test_candidate_change_before_apply_rejects`
Single-use manager-bound plans	`test_manager_bound_single_use_handle`; `test_cross_manager_handle_isolation`
Whole-pair verification	`test_shared_guardrail_canary_fault_requires_full_pair_restart`
No unnecessary response on matching probes	`test_healthy_scores_do_not_trigger_repair`
Traffic warning does not consume a later runtime response	`test_early_traffic_warning_does_not_consume_later_runtime_response`
New incident after hold release	`test_release_then_refault_opens_a_new_incident_without_healthy_tick`
Missing current output can use independently verified reload	`test_failed_current_samples_still_allow_independently_verified_restart`
Containment needs separate approval	`test_containment_requires_separate_approval_and_no_fallthrough`
Exceptions exclude private request text	`test_backend_error_redaction_without_private_word`

Validation scope and next reliability gates

- Recovery restores the tested baseline. It does not make that baseline semantically correct on every request.
- Finite probes can miss input-specific faults or an attacker who avoids the panel.
- Both classifiers share data and one estimator family. Healthy synthetic error correlation is **0.7888**.
- They are both wrong on **2,033 / 20,000** healthy synthetic test requests. Canary agreement is not a safety label.
- Live labels can be absent. Separate labelled evaluation remains necessary for quality claims.
- Offline approval flags are implemented; production identity, approval separation, and durable incident services are **Not implemented**.
- The supervisor bounds incidents at 128. Direct manager usage retains consumed verification payloads; long-lived memory qualification remains open.
- Digests detect changed bytes. They are not signatures, hardware attestation, or proof of semantic correctness.

Recommended visual

Use a horizontal decision gate: **Candidate → Execute → Verify pair → Reverify → Activate**. Add a lower path from any unsuccessful verification to **Explicit hold**. Use a separate small box for the earlier conformal experiment so it does not imply current automatic recalibration.

Evidence

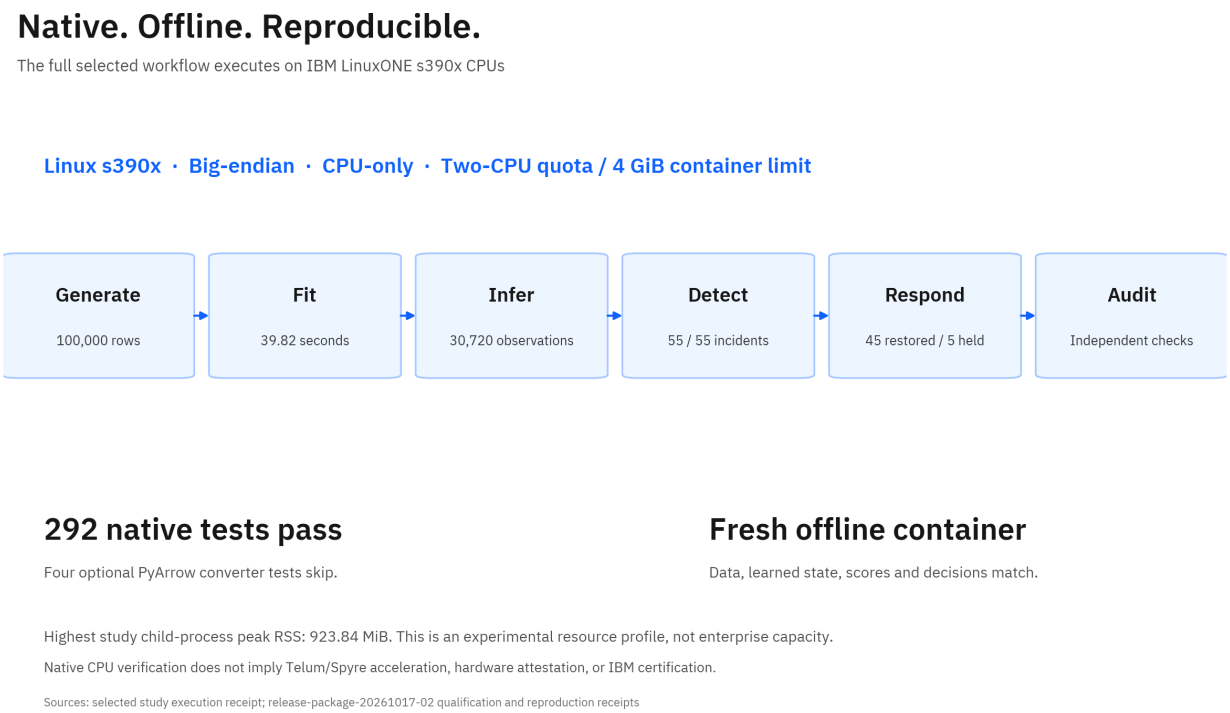
[C1], [C2], [C3], [C11]; [E1], [E8], [E10]; the exact named tests in `tests/test_baseline_recovery.py` and `tests/test_runtime_supervision.py`.

Speaker notes

“Our strongest reliability feature is the activation gate. A plan is not enough; the candidate must execute and pass fresh verification. We preserve the original policy during runtime recovery. A contained service is shown as contained, not as repaired or correctly classified.”

SLIDE 10

Native IBM Z Execution, Evidence-Backed Control



Main message

Veyra demonstrates a complete guardrail-integrity workflow on the native platform where enterprise Linux AI can run.

Slide points

- Run data generation, model fit, inference, checks, recovery, and audit natively on LinuxONE.
- Reproduce the selected workflow in a fresh container with networking disabled.
- Keep model artifacts and request data in the private execution environment.
- Give platform, safety, and model-risk teams a traceable response decision.

Why LinuxONE matters to this project

Architecture compatibility is demonstrated, not assumed. The implementation runs on big-endian `s390x`, not x86 emulation. The native environment supports the selected scikit-learn pipeline and the archived PyTorch/Transformers comparison.

The control loop stays with the inference workflow. Once required artifacts are present, the selected experiment runs without an external model API. The offline qualification explicitly disables container networking. This supports a private-deployment design direction; it does not establish a complete customer residency or compliance program.

The resource profile is concrete. The qualified container uses a two-CPU quota and 4 GiB memory limit. The selected study records 923.84 MiB as its highest child-process peak RSS. This is an actual development profile, not enterprise capacity sizing.

Reproducibility is part of the platform result. The private kit contains source, a wheel, frozen models and data, audit records, and the native runtime image. The reproduction matches all data bytes, learned model state, request scores, decisions, response actions, and benchmark metrics. It also produces 156 byte-identical algorithm files. Serialized model bytes can differ because of scikit-learn's process-local `_stop_words_id` cache. The distributed model file retains its original fixed hash.

Verified platform facts

Item	Verified state
Operating target	LinuxONE Community Cloud; Ubuntu Linux; `s390x`
Byte order	Big-endian
Python	3.12.15
NumPy / SciPy	2.0.2 / 1.13.1
scikit-learn / joblib	1.9.0 / 1.6.0
PyTorch / Transformers comparison environment	2.11.0+cpu / 5.19.0
Selected runtime	CPU only
Fresh-container network	Disabled during qualification
Model/data source mount	Read-only during qualification
Native test outcome	292 pass; four optional PyArrow skips

Platform scope

Telum acceleration, Spyre acceleration, Secure Execution, hardware attestation, z/OS integration, and HA are **Not measured or implemented here**. The project does not claim IBM certification or a LinuxONE speed advantage over another architecture. LinuxONE provides the verified native execution target. It does not itself improve classifier accuracy.

Practical users and applications

Target user	Potential use	Current evidence
AI platform engineer	Validate a guardrail wrapper or model-runtime change	Native paired fault replay and artifact checks
LinuxONE platform operator	Qualify a local CPU safety-control workflow	Native container, dependency, resource, and reproduction evidence
Trust and safety analyst	Separate traffic warnings from runtime incidents	Distinct signal channels and explicit event records

Target user	Potential use	Current evidence
Model-risk reviewer	Examine a proposed recovery and its decision effects	Paired verification, approval records, and full-denominator metrics

These are target workflows, not confirmed customers or measured business savings. Customer interviews, adoption, ROI, and commercial rights clearance are **Not measured or completed**.

Strong closing line

“A guardrail needs a watcher. On LinuxONE, we make that watcher measurable—and every recovery verifiable.”

Recommended visual

Use `visuals/native-proof.png`: **Generate → Fit → Infer → Detect → Verify → Audit**, inside one LinuxONE boundary. End with the closing line and the public release branch URL.

Evidence

[E1]–[E6], [C8], [C12], [P1], [I1]. Public release:
https://github.com/beastgotfried/agent-jailbreak/tree/release/safety-watch-baseline-20261017

Speaker notes

“LinuxONE is the execution foundation of this project. We verify the actual native CPU workflow, including model fit and recovery. The private kit repeats the result offline. Our contribution is a measurable response loop around guardrails, with evidence for each transition.”

APPENDIX A

Strongest Features to Highlight

1. **Native, end-to-end LinuxONE execution.** Model fit and actual inference occur on `s390x`; the server is not only a storage location.
2. **Actual candidate verification.** Candidate execution and fresh activation checks control the backend switch.
3. **Separation of evidence and authority.** A traffic warning cannot automatically authorize recovery.
4. **Complete-stream evaluation.** The score includes response delay and gives holds no correctness credit.
5. **Repeated incident handling.** A new runtime fault can open a new incident after recovery or release.
6. **Explicit containment.** The system distinguishes a held service from restored service.
7. **Whole-pair checks.** A guardrail-only correction cannot conceal a changed canary.
8. **Reproducible private artifacts.** Exact source, model identities, input hashes, and numeric audits support the release.
9. **Measured model choice.** The selected release retains the stronger completed operational candidate.
10. **Practical CPU execution.** The selected study fits in the declared native resource profile without an external model API.

Safe headline language

Use	Keep this scope beside it
“55 of 55 runtime incidents confirmed”	Controlled fault suite; not universal attack detection
“45 fault episodes restore service; five remain contained”	50 fault episodes plus ten controls
“87.44% fewer harmful allows”	Synthetic labels; same paired request stream; includes containment effects
“81.69% complete-stream operational score”	Holds count as zero; not production safety accuracy
“Native LinuxONE reproduction”	Qualified CPU environment; not hardware attestation
“Verified recovery”	Agreement on finite probes and the measured endpoint; not universal semantic repair

APPENDIX B

Feature Status Matrix

Feature	Status	Evidence or next step
Frozen TF-IDF guardrail and canary	Implemented and measured	Model cards and selected manifest
Native model fit and inference	Implemented and measured	Execution receipts and fresh-container reproduction
Binary decision policy	Implemented	`BinaryPolicy`
Classifier CLI	Implemented	`safety-watch inspect/score`
Runtime matched probes	Implemented and measured	`RuntimeWatch`; 55 confirmed incidents
Finite-horizon KS monitor	Implemented and measured in separate studies	`DisagreementMonitor`
Source-bound traffic profile	Implemented; finite study scope	Dolly profile report
Conformal response	Implemented; candidates do not pass the fixed historical gate	Eight rejected applications; no claimed improvement
Affine baseline correction	Implemented and measured	Scale and affine traces
Trusted backend reload	Implemented and measured	Square, constant, common-mode, and other traces
Explicit containment and hold release	Implemented; release paths also have fixture tests	`SupervisedRuntime`
Generation-bound single-use plans	Implemented and tested	Recovery API tests
Repeated runtime incidents	Implemented and measured	`scale_then_constant`
Runtime output unavailable path	Implemented and fixture-tested	Restart-only and containment tests
Local evidence and audit files	Implemented	JSON/JSONL, hashes, independent audit
Approval workflow	Partially implemented	Offline flags and actors; no production authentication
Streaming service	Partially implemented	Ordered local streams; no live collector or broker
Product dashboard	Planned; Not implemented	PRD console map only
REST API, Kafka, PostgreSQL, S3	Planned; Not implemented	PRD architecture only
Customer connectors and autonomous agent integration	Planned; Not implemented	Current execution target is an offline stub
Scan martingales, MMD	Research reference; Not implemented	Present in Cheap Canaries, not this release
Hardware attestation, Telum/Spyre acceleration	Not implemented or measured	Separate platform qualification required

Feature	Status	Evidence or next step
Active accuracy extension	In progress; outside this release	Protocol at `323c2b0`; no result imported into this document

APPENDIX C

Asset and Evidence Inventory

New presentation assets supplied with this document

File	Purpose	Provenance
visuals/architecture.png and .svg	Implemented architecture	Source-backed diagram, not a UI screenshot
visuals/architecture.mmd	Editable Mermaid architecture	Same implemented components and boundaries
visuals/runtime-recovery.png and .svg	Fault-to-recovery timeline	Actual scale/seed-0 request score errors; no smoothing
visuals/stream-outcomes.png and .svg	Complete-stream comparison	Exact counts from the selected report
visuals/response-outcomes.png and .svg	Restored, held, and control episodes	Exact 60-episode outcomes
visuals/data-split.png and .svg	Train/validation/test structure	Selected data manifest
visuals/native-proof.png and .svg	LinuxONE execution proof chain	Native study and release receipts
tables/complete-stream-results.csv	Editable comparison data	Same denominators as Slide 8
tables/scenario-results.csv	Editable condition table	All 12 conditions, five seeds each
tables/classifier-results.csv	Classifier quality charts	Selected guardrail and canary; three evaluation sources
tables/*-timeline.csv	Scale, square, persistent, and recurrent traces	Derived from hash-verified native event files
evidence/demo-cases.json	Exact demo transitions	Source file hashes and endpoint checks
evidence/metrics.json	Structured pitch metrics	Selected fields from frozen aggregate reports
evidence/code-index.json	Function and source-position index	44 Python files in the release snapshot
evidence/native-platform-snapshot.json	Native architecture and resource evidence	Read-only platform query; no new model experiment
evidence/source-manifest.json	Source/receipt hash index	Frozen release, core, study, and prior-art identities

Existing visuals and recordings

Existing asset	Availability	Presentation use
Repository `docs/img/architecture.png`	Available	Existing overview; prefer the new implementation-only diagram for this pitch
`prd/Veyra-Architecture-v1.png`	Available	Enterprise proposal; label it planned if used
`prd/Veyra-Enterprise-PRD-v1.0.pdf`	Available	Product requirements reference, not delivered-feature proof
`prd/previews/*.png` and `contact-sheet.png`	Available	PRD page previews; not product screenshots
Native pytest and qualification logs	Available	Capture terminal proof after sensitive fields are excluded
Private Jupyter environment	Available development tool	A new screenshot can show real execution; it is not a product console
Product frontend screenshots	Not available	Frontend is not implemented
Finished live-demo recording	Not found in the inspected authorized project files	Record before submission
Finished product notebooks	Not found in the inspected project inventory	Do not imply a delivered notebook application

Completed experiment records

Study	Available evidence	Correct interpretation
Original ToxicChat classifiers	Model cards, split manifest, execution receipt	Separate original models; not the selected synthetic-trained pair
Original prompt pools	Pool manifest, exclusions, independent provenance audit	4,352 normalized groups and 5,032 source links
Original KS calibration	Calibration artifact, score manifest, tests	Finite-horizon empirical threshold
Original conformal replay	15 paired episodes; 7,680 events	Eight candidates blocked by verification; no applied improvement
Historical baseline recovery	50 episodes; 35/40 fault restorations	Earlier result includes timing/reference defects; not the current causal headline
Repaired original-model qualification	60 episodes; 11,520 events; independent audit	55 causal confirmations; 45 restored; five held
Dolly traffic profile	Five 512-event episodes; 145 KS windows independently checked	Two warning episodes; no safety-label performance claim
Selected synthetic study	100,000 rows; 60 episodes; 30,720 observations	Current completed release basis
Neural comparison	60 episodes; 30,720 observations; 12-condition seed-0 repeat	Archived comparison; 36 repeated files; not a second full independent trial

Study	Available evidence	Correct interpretation
Release reproduction	Fresh offline native container; data/refit/replay/audit	Same learned state and results; reproducibility, not new effectiveness evidence

The original synthetic repeat matches 182 algorithm files. The release reproduction matches 156 files byte-for-byte and checks all request scores, decisions, response actions, and metrics. Model-bound identifiers can change after a refit because serialized cache metadata can change.

APPENDIX D

Dataset Attribution and Label Boundaries

Source links and immutable revisions

Source	Revision or snapshot identity	Link
ToxicChat	`29df8e4dba60e1f4af4b4075c0705c5b313548a8`	https://huggingface.co/datasets/lmsys/toxic-chat
WAInjectBench	`001df8e431969089812df6694dfa607fe7be79e8`	https://huggingface.co/datasets/Norrrrrrr/WAInjectBench
JBB-Behaviors	`886acc352a31533ffbcf4ef22c744658688086fc`	https://huggingface.co/datasets/JailbreakBench/JBB-Behaviors
JailbreakBench mirror	`119a73e012e44035957c25a483f7f721bb348839`	https://huggingface.co/datasets/walledai/JailbreakBench
StrongREJECT	`9f4b3397bda985e1ac6dadb655e9ca3a17974144`	https://huggingface.co/datasets/AlignmentResearch/StrongREJECT
HarmBench	Hash-pinned `main@2026-10-16` snapshot, not an immutable revision	https://github.com/centerforaisafety/HarmBench
XSTest	`d7bb5bd738c1fcbc36edd83d5e7d1b71a3e2d84d`	https://github.com/paul-rottger/xstest
Dolly 15k	`5f9bfba300181fb35854d64c8bf1180ea75105f7`	https://github.com/databrickslabs/dolly
OASST1	`fdf72ae0827c1cda404aff25b6603abec9e3399b`	https://huggingface.co/datasets/OpenAssistant/oasst1

The complete HarmBench CSV hash is `8d81accedd38eaaf8b760618622bb888417d1fd0c86eba65c427a16f1cbb4afc`. The synthetic catalogue hash is `852370c8b0edca168aeaaf1be845813429595d1bb49550cfdd887b874f1d53eb`.

Task-specific label definitions

- **Synthetic study:** Catalogue intent, safe 0 or harmful 1; matched pairs stay together.
- **ToxicChat:** Harmful label derives from toxicity OR jailbreaking; human annotation is reported separately.
- **WAInjectBench:** Source prompt-injection routing labels; not universal harmful-request labels.
- **JBB:** Official `Goal` field; never the target response field.
- **StrongREJECT:** `clf_label` governs the mapped request label; proxy labels stay separate.
- **HarmBench:** Only standard behavior requests enter the original pool.
- **XSTest:** Source request labels; family-based verification/evaluation separation.
- **Dolly and OASST1:** No safety labels enter the traffic-profile work described here.

Dolly splits are 5,897 reference / 4,423 calibration / 4,424 evaluation groups. The original prompt-pool partitions are 1,256/0 reference, 942/604 calibration, and 945/605 evaluation, in safe-routing/attack-routing order.

Active work outside the frozen release

The accuracy protocol names `nvidia/Aegis-AI-Content-Safety-Dataset-2.0`, revision `d86bb8bedff51d25ac834ab7838f1cc61acb7a2c`, CC-BY-4.0. It proposes human `prompt_label` data and preserves official split boundaries. It is not a source for the selected release metrics in this document. Completed fit and result evidence are not imported from that active branch.

Sources not used for selected-model or completed benchmark claims

WildGuardMix, SafePickle, LongPIBench, WildChat-1M, LMSYS-Chat-1M, and the gated walledai HarmBench/StrongREJECT releases do not enter these claims. OR-Bench is a paper reference, not an acquired evaluation source here. WhyLabs pattern groups serve as reference material, not as the selected classifier or runtime.

Pretrained model credit

- BERT-tiny: `google/bert_uncased_L-2_H-128_A-2`, revision `30b0a37ccaaa32f332884b96992754e246e48c5f`, Apache-2.0; archived comparison.
- MiniLM-L6: `sentence-transformers/all-MiniLM-L6-v2`, revision `1110a243fdf4706b3f48f1d95db1a4f5529b4d41`, Apache-2.0; native compatibility/cost check only.

Public access is not an unrestricted license. ToxicChat has a noncommercial restriction; several inspected sources have no verified dataset license. Commercial data rights and redistribution approval remain separate from execution and package qualification.

APPENDIX E

Most Compelling Three-Minute Demonstration

Recommended sequence

Time	Show	Say	Evidence status
00:00–00:15	Product name and one-line architecture	“The guardrail checks requests. Veyra checks the guardrail runtime.”	Implemented components
00:15–00:35	Native terminal architecture and version output	“This is native LinuxONE s390x CPU execution, not emulation.”	Capture live; exclude connection details
00:35–00:55	`safety-watch inspect`, then a neutral JSONL scoring call	“We verify the selected artifact and execute the real model pair.”	Classifier-only; no claim of full monitoring in this command
00:55–01:35	Scale/seed-0 trace: 128 → 143 → 175	“The output changes. Probes confirm it. A verified correction restores the tested baseline.”	Label chart as recorded native replay unless the episode runs live
01:35–02:00	Recurrent trace or square case	“The next fault gets a new incident; a non-affine case uses a verified backend reload.”	Recorded native trace or prepared live wrapper
02:00–02:25	Persistent-fault hold result	“The replacement does not pass verification, so the system keeps an explicit hold.”	Hold is not restoration
02:25–02:50	Full-denominator outcome chart and 292-pass test receipt	“We audit the complete stream and repeat the pipeline in a fresh offline native container.”	Completed qualification
02:50–03:00	Closing line and release URL	“A guardrail needs a watcher. On LinuxONE, every recovery has evidence.”	Product conclusion

Ready native commands

Run these inside the qualified native environment with the private bundle mounted at `/bundle`:

```
python -c 'import platform,sys; print(platform.machine(), platform.python_version(), sys.byteorder)'
safety-watch inspect --models /bundle/studies/synthetic-100k-20261017-01/models
printf '%s\n' '{"text":"Explain the purpose of a software health check."}' | safety-watch score
  --models /bundle/studies/synthetic-100k-20261017-01/models
```

The neutral prompt is a demonstration input, not a labelled benchmark example. Show its actual output. Do not preclaim an allow decision or classifier correctness for that input.

Live fault-replay preparation

The existing `episode()` function supports one named condition and seed. A small demo wrapper can load the frozen test/probe records, select `scale`, seed 0, and call that existing function. This document does not claim that a timed three-minute wrapper or recording already exists. The standard replay CLI runs all 60 episodes; its measured 191.64-second stage does not fit beside other content in a three-minute video.

For a credible submission, either prepare and time that bounded wrapper or clearly identify the fault segment as a recorded native replay. Never animate fabricated output and describe it as a live run.

Demo preparation checklist

- Confirm the bundle and runtime image hashes before recording.
- Use a new output directory; preserve the frozen study outputs.
- Keep requests as data; show IDs or aggregate results rather than raw harmful text.
- Hide server addresses, credentials, tokens, and private filesystem details.
- Label the classifier-only CLI and full-runtime replay separately.
- Keep a local copy of the verified trace and charts as a recording fallback.
- Time the complete recording to no more than three minutes.

APPENDIX F

Measurements and Evidence to Collect Before Submission

Priority	Item	Current status	Why collect it
Essential	Final three-minute native recording	Not available in the inspected files	Show actual LinuxONE execution
Essential	One-condition live replay wrapper and elapsed time	Not prepared or timed in this extraction	Keep a live fault demonstration within the video limit
Essential	Final source/receipt snapshot after active work finishes	Frozen release is available; accuracy branch remains separate	Avoid mixing unfinished results with release claims
Essential	Submission and dataset-rights review	Not completed by these experiments	Confirm event rules and permitted use
Next qualification	Fresh policy-aligned, untouched external evaluation	Not measured for the selected release	Establish external generalization beyond exposed regressions
Next qualification	p50/p95/p99 request and response latency	Not measured	Measure end-to-end behavior, not score-call sums
Next qualification	Throughput, concurrent streams, CPU utilization, and sustained memory	Not measured	Establish capacity and long-lived behavior
Next qualification	Runtime false confirmations per hour under representative traffic	Not measured	Episode controls do not establish production false-page rates
Next qualification	Selective faults outside the probe panel and probe-aware attacks	Fixture counterexample exists; broad rate not measured	Quantify probe coverage
Next qualification	Shared semantic-error coverage and delayed-label workflow	Correlation measured; operational label workflow not implemented	Separate integrity from correctness
Next qualification	Long-lived issuance-store retention	Known retained consumed payloads; sustained profile not measured	Qualify direct manager use beyond bounded episodes
Enterprise roadmap	Product UI, authenticated API, durable collector, database, broker	Not implemented	Support a deployed operator workflow
Enterprise roadmap	Customer enforcement connector and agent integration	Not implemented	Move beyond the offline stub
Enterprise roadmap	HA, disaster recovery, tenant isolation, security review	Not measured or implemented	Qualify enterprise operation

Priority	Item	Current status	Why collect it
Optional platform study	LinuxONE versus x86 performance, energy, or cost	Not measured	Support comparative hardware claims
Optional platform study	Telum/Spyre or Secure Execution integration	Not implemented or measured	Add processor-specific capability evidence
Product discovery	Interviews, design partners, adoption, ROI	Not measured	Validate demand and commercial value

APPENDIX G

Source and Evidence References

Code references

All release code links use commit `8a86ce01767a02d613b66f38a1d20de312cf61e1`. Function line positions also appear in `evidence/code-index.json`.

- **[C1] Supervised runtime:** `src/response/supervised.py`; `SupervisedRuntime._respond`, `step`, `release_hold`. https://github.com/beastgotfried/agent-jailbreak/blob/8a86ce01767a02d613b66f38a1d20de312cf61e1/src/response/supervised.py
- **[C2] Recovery authority and verification:** `src/response/recovery.py`; `fit_affine`, `verify_outputs`, `RecoveryManager.propose`, `propose_restart`, `apply`, `rollback`. https://github.com/beastgotfried/agent-jailbreak/blob/8a86ce01767a02d613b66f38a1d20de312cf61e1/src/response/recovery.py
- **[C3] Runtime checks:** `src/monitor/runtime.py`; `checked_scores`, `HealthProbe`, `RuntimeWatch.tick`, `verified_release`. https://github.com/beastgotfried/agent-jailbreak/blob/8a86ce01767a02d613b66f38a1d20de312cf61e1/src/monitor/runtime.py
- **[C4] Passive KS monitor:** `src/monitor/disagreement.py`; `ScorePair.gap`, `ks_statistic`, `calibrate`, `DisagreementMonitor.update`. https://github.com/beastgotfried/agent-jailbreak/blob/8a86ce01767a02d613b66f38a1d20de312cf61e1/src/monitor/disagreement.py
- **[C5] Selected models and thresholds:** `src/models/synthetic.py`; `guard_features`, `train`, `select_threshold`, `BinaryPolicy`, `SyntheticBackend`. https://github.com/beastgotfried/agent-jailbreak/blob/8a86ce01767a02d613b66f38a1d20de312cf61e1/src/models/synthetic.py
- **[C6] Full paired benchmark:** `src/replay/synthetic.py`; `episode`, `service_metrics`, `combine`, `external_scores`, `run`. https://github.com/beastgotfried/agent-jailbreak/blob/8a86ce01767a02d613b66f38a1d20de312cf61e1/src/replay/synthetic.py
- **[C7] Data generation and splits:** `src/data/synthetic.py`; `family_splits`, `validation_roles`, `expand`, `generate`. https://github.com/beastgotfried/agent-jailbreak/blob/8a86ce01767a02d613b66f38a1d20de312cf61e1/src/data/synthetic.py
- **[C8] Selected release interface:** `src/release/cli.py`; `verify_selected`, `score_lines`, `main`; identity in `src/release/selected.json`. https://github.com/beastgotfried/agent-jailbreak/blob/8a86ce01767a02d613b66f38a1d20de312cf61e1/src/release/cli.py
- **[C9] Source-bound traffic profiles:** `src/monitor/profiles.py` and `src/monitor/qualify_profile.py`. https://github.com/beastgotfried/agent-jailbreak/blob/8a86ce01767a02d613b66f38a1d20de312cf61e1/src/monitor/profiles.py
- **[C10] Fault transforms and earlier causal replay:** `src/replay/recovery.py::FaultBackend`; `src/replay/continuous.py::run_episode`. https://github.com/beastgotfried/agent-jailbreak/blob/8a86ce01767a02d613b66f38a1d20de312cf61e1/src/replay/recovery.py
- **[C11] Historical conformal response:** `src/response/conformal.py`; `fit_policy`, `verify_candidate`, `approved_recalibration`. https://github.com/beastgotfried/agent-jailbreak/blob/8a86ce01767a02d613b66f38a1d20de312cf61e1/src/response/conformal.py
- **[C12] Reproduction and release identity:** `scripts/reproduce_selected.py`, `scripts/package_private.py`, `src/release/state.py`. https://github.com/beastgotfried/agent-jailbreak/blob/8a86ce01767a02d613b66f38a1d20de312cf61e1/scripts/reproduce_selected.py

Experiment references

Paths below are relative to the authorized `ibm-z` project root. They are private evidence locations, not public downloads. Their hashes appear in `evidence/source-manifest.json`.

- **[E1] Selected benchmark:** `artifacts/synthetic-100k-20261017-01/benchmark-report.json`; fields `overall`, `aggregate`, `healthy_test`, `external_regression`, `runtime_model_inputs`, `error_correlation`.
- **[E2] Data and selected models:** `artifacts/synthetic-100k-20261017-01/data-manifest.json` and `model-cards.json`.
- **[E3] Selected native resource evidence:** `artifacts/synthetic-100k-20261017-01/execution-receipt.json` and `timing.json`.
- **[E4] Release qualification:** `artifacts/release-package-20261017-02/qualification-receipt.json`, `reproduction-receipt.json`, `native-tests.log`, `independent-audit.json`; read-only environment data in `evidence/native-platform-snapshot.json`.
- **[E5] Neural comparison:** `artifacts/neural-upgrade-20261017/benchmark-report.json`, `model-cards.json`, `execution-receipt.json`, `baseline-comparison.json`, `repeatability.json`.
- **[E6] Original real-data models:** `artifacts/classifiers-20261017-01/model-cards.json`, `split-manifest.json`, `execution-receipt.json`.
- **[E7] Original pools and source records:** `artifacts/prompt-pools-20261017-01/pool-manifest.json`; `preparation/prompt-pools-20261017/source-download-manifest.json`; public `releases/baseline-20261017/data-sources.json`.
- **[E8] Repaired runtime and Dolly study:** `artifacts/repair-loop-20261017/qualification-1/independent-audit.json`, `continuous-report.json`, `traffic-profile-report.json`, `repeatability.json`.
- **[E9] KS and reference diagnostics:** `artifacts/monitor-20261017-01/score-manifest.json`; `artifacts/monitor-diagnostic-20261017-01/investigation.json`; original calibration and source commit `f7ce7c8129b80c9f37edfa7a903769b0238cbfaa`.
- **[E10] Original conformal gate:** `artifacts/replay-20261017-01/response-verification-summary.json` and `scorecard.md`; exact historical source remains in Git history/private evidence.

Public aggregate results: https://github.com/beastgotfried/agent-jailbreak/blob/8a86ce01767a02d613b66f38a1d20de312cf61e1/releases/baseline-20261017/summary.json

Prior art

The source review pins Cheap Canaries to commit `90726d9d4d281373e6729a0319d50e9c32566c59`. The repository and paper results belong to their authors. This extraction does not rerun or certify their experiments.

- **[R1] Paper and repository:** *Cheap Canaries / Online Shift Detection and Conformal Adaptation for Deployed Safety Classifiers*. https://arxiv.org/abs/2606.11949 ; https://github.com/junwenleong/safety-classifier-shift-monitor
- **[R2] Reference KS:** `shift_detection_monitor/detection/ks_detector.py::KSDetector`. https://github.com/junwenleong/safety-classifier-shift-monitor/blob/90726d9d4d281373e6729a0319d50e9c32566c59/shift_detection_monitor/detection/ks_detector.py
- **[R3] Reference scan martingale:** `shift_detection_monitor/detection/conformal_martingale.py::ScanMartingale`, `_conformal_p_two_sided`, `_bet_log_increment`. https://github.com/junwenleong/safety-classifier-shift-monitor/blob/90726d9d4d281373e6729a0319d50e9c32566c59/shift_detection_monitor/detection/conformal_martingale.py
- **[R4] Reference MMD and adaptation:** `detection/mmd_detector.py`, `adaptation/conformal.py`, `adaptation/density_ratio.py`. https://github.com/junwenleong/safety-classifier-shift-monitor/tree/90726d9d4d281373e6729a0319d50e9c32566c59/shift_detection_monitor

The reference also contains CUSUM/EWMA and other research detectors. Do not claim our smaller method set exceeds its statistical scope. A broader product-level originality or market-first claim requires a separate competitive review.

Product and IBM references

- **[P1] Product proposal:** `prd/Enterprise-PRD.md`, version 1.0, 16 October 2026. It describes enterprise requirements, not delivered features. Its initial environment-only status predates the completed studies.
- **[I1] IBM platform context:** https://www.ibm.com/products/linuxone ; https://github.com/linuxone-community-cloud/jupyter-lab-ml ; https://community.ibm.com/zsystems/l1cc/

IBM product capabilities provide context. Only the native CPU workflow and recorded environment checks support this project's implementation claims.

APPENDIX H

Design and Editorial Direction

Visual system

- Use IBM Plex Sans for titles and body text.
- Use IBM Plex Mono for source commits, event indices, and native platform output.
- Use dark ink, white space, IBM blue, and a restrained teal accent.
- Use one main diagram or chart per slide.
- Keep main-slide text to the four supplied points.
- Put units, denominators, and evidence scope directly below each chart.
- Use labels as well as color for restored, held, and control states.
- Use actual evidence charts rather than stock cybersecurity imagery or fictional dashboards.

Suggested main-slide asset order

1. Product title and simplified architecture.
2. Same request, changed score path illustration.
3. Three evidence/authority lanes.
4. Implemented native architecture.
5. Dataset split and model pair.
6. Separate KS and matched-probe flows.
7. Actual scale-fault timeline.
8. Complete-stream outcomes and fault-episode counts.
9. Candidate verification gate.
10. Native proof chain and closing line.

Final editorial check

Positive presentation comes from demonstrated capabilities, not omitted denominators. Keep the PRD roadmap, research methods, model quality, runtime restoration, and hardware scope distinct. The strongest pitch is the one the code and receipts can reproduce.